

图论强化训练

单源最短路径专项练习（Python）

涵盖内容：

- ✓ 无权图最短路径：BFS
- ✓ 有向无环图最短路径：DAG Relaxation
- ✓ 含负权边图：Bellman-Ford
- ✓ 非负权图：Dijkstra + 优先队列
- ✓ 松弛操作、前驱记录与路径恢复
- ✓ 不可达顶点与负权环处理

语言：Python 请先完成引导性问题，再补全程序并运行测试

预备知识：单源最短路径与松弛操作

设带权图为 $G = (V, E)$ ，边 (u, v) 的权值为 $w(u, v)$ ，指定源点为 s 。单源最短路径问题要求计算从 s 到所有顶点 $v \in V$ 的最短距离 $\delta(s, v)$ 。

四种算法虽然适用条件不同，但都围绕松弛操作展开。若当前已经得到距离估计值 $d[u]$ ，则对边 (u, v) 执行

$$\begin{cases} d[v] \leftarrow d[u] + w(u, v), & \text{当 } d[u] + w(u, v) < d[v] \text{ 时。} \\ \text{parent}[v] \leftarrow u, \end{cases} \quad (1)$$

其中， $d[v]$ 是当前最短距离估计， $\text{parent}[v]$ 记录当前最短路径中顶点 v 的前驱。

四种算法的适用范围如下：

算法	适用条件	时间复杂度
BFS	无权图或所有边权相同	$O(V + E)$
DAG Relaxation	有向无环图，允许任意权值	$O(V + E)$
Bellman-Ford	一般图，允许负权边	$O(V E)$
Dijkstra	所有边权非负	$O((V + E) \log V)$

第 1 题 无权图的单源最短路径（BFS）

考点：队列 · 分层遍历 · 距离数组 · 前驱数组 · 路径恢复

1.1 背景知识

在无权图中，可以把每条边的权值视为 1。从源点 s 到顶点 v 的最短距离，就是从 s 到 v 所需经过的最少边数。

BFS 按照距离源点由近到远的顺序分层访问顶点。当一个顶点第一次被发现时，其最短距离已经确定：

$$\begin{cases} d[s] = 0, \\ d[v] = d[u] + 1, & \text{当 } v \text{ 第一次由 } u \text{ 访问时。} \\ \text{parent}[v] = u, \end{cases} \quad (2)$$

由于每个顶点最多入队一次，每条边最多被检查常数次，BFS 的时间复杂度为 $O(|V| + |E|)$ 。

1.2 引导性问题

1. 为什么 BFS 能够保证顶点第一次被访问时得到的就是最短距离？队列的先进先出性质起到了什么作用？

2. 若使用栈代替队列，算法会变成什么遍历？此时还能保证得到无权图最短路径吗？
3. 如何表示源点无法到达的顶点？使用 `float("inf")` 有什么优点？
4. 如果在顶点出队时才将其标记为已访问，同一顶点是否可能被重复加入队列？
5. 已知 `parent` 数组后，如何从目标点逆向恢复路径？为什么最后需要将路径翻转？

1.3 题目要求

实现以下方法：

1. `add_edge(u, v)`：添加无向边。
2. `shortest_paths(source)`：用 BFS 计算距离和前驱。
3. `reconstruct_path(source, target, parent)`：恢复源点到目标点的最短路径；不可达时返回空列表。

1.4 接口参考（仅声明，补全实现）

```
1 from collections import deque
2 from typing import List, Tuple
3
4 class UnweightedGraph:
5     def __init__(self, n: int):
6         self.n = n
7         self.adj: List[List[int]] = [[] for _ in range(n)]
8
9     def add_edge(self, u: int, v: int) -> None:
10         """添加无向边 (u, v)。"""
11         pass # TODO
12
13     def shortest_paths(
14         self, source: int
15     ) -> Tuple[List[float], List[int]]:
16         """
17         返回：
18         dist[i]: source 到 i 的最少边数；
19         parent[i]: 最短路径中 i 的前驱。
20         不可达顶点的距离为 inf，前驱为 -1。
21         """
22         pass # TODO
```

```
23
24     @staticmethod
25     def reconstruct_path(
26         source: int, target: int, parent: List[int]
27     ) -> List[int]:
28         """恢复 source 到 target 的路径，不可达则返回 []。"""
29         pass # TODO
```

1.5 测试用例

```
1 if __name__ == "__main__":
2     # 情形 1: 简单连通图
3     # 0 -- 1 -- 3 -- 4
4     # |   |
5     # 2 --+
6     # 顶点 5 与其他顶点不连通
7     g = UnweightedGraph(6)
8     g.add_edge(0, 1)
9     g.add_edge(0, 2)
10    g.add_edge(1, 2)
11    g.add_edge(1, 3)
12    g.add_edge(3, 4)
13
14    dist, parent = g.shortest_paths(0)
15
16    print(dist) # [0, 1, 1, 2, 3, inf]
17    print(g.reconstruct_path(0, 4, parent))
18    # 可能输出 [0, 1, 3, 4]
19
20    print(g.reconstruct_path(0, 5, parent))
21    # []
22
23    # 情形 2: 源点等于目标顶点
24    dist2, parent2 = g.shortest_paths(1)
25    print(dist2[1]) # 0
26    print(g.reconstruct_path(1, 1, parent2)) # [1]
27
28    # 情形 3: 单顶点
29    g3 = UnweightedGraph(1)
```

```
30 dist3, _ = g3.shortest_paths(0)
31 print(dist3) # [0]
```

第 2 题 有向无环图最短路径（DAG Relaxation）

考点：拓扑排序 · 入度 · Kahn 算法 · 任意权值 · 按序松弛

2.1 背景知识

若图是有向无环图（DAG），则一定存在拓扑序 $v_1, v_2, \dots, v_n$ ，使每条边都从拓扑序中较早的顶点指向较晚的顶点。

按照拓扑序依次松弛每个顶点的所有出边，可以保证：处理顶点 u 时，所有能够到达 u 的前驱均已处理，因此 $d[u]$ 已经是最终最短距离。

算法可以表示为

$$\begin{cases} \text{求图的拓扑序 } v_1, v_2, \dots, v_n, \\ \text{依次对 } v_i \text{ 的所有出边执行松弛操作。} \end{cases} \quad (3)$$

DAG 最短路径允许负权边，因为拓扑序保证算法不会反复绕环。其时间复杂度为 $O(|V| + |E|)$ 。

2.2 引导性问题

1. 为什么有向无环图一定存在拓扑序？若图中存在有向环，Kahn 算法最终会出现什么现象？
2. 每添加一条边 $u \rightarrow v$ ，应当增加哪个顶点的入度？
3. 为什么 DAG 最短路径允许负权边，而 Dijkstra 不允许？
4. 若某顶点 u 从源点不可达，即 $d[u] = +\infty$ ，是否还需要松弛它的出边？为什么？
5. 拓扑序是否唯一？采用不同的合法拓扑序，最终得到的最短距离是否相同？
6. 若拓扑排序处理的顶点数少于 n ，程序应当如何处理？

2.3 题目要求

实现以下方法：

1. `add_edge(u, v, weight)`：添加有向带权边，并更新入度。
2. `topological_sort()`：使用 Kahn 算法生成拓扑序；若图中存在环，则抛出 `ValueError`。
3. `shortest_paths(source)`：按照拓扑序松弛所有边。
4. `reconstruct_path(...)`：根据前驱数组恢复最短路径。

2.4 接口参考（仅声明，补全实现）

```
1 from collections import deque
2 from typing import List, Tuple
3
4 class DAGShortestPath:
5     def __init__(self, n: int):
6         self.n = n
7         self.adj: List[List[Tuple[int, float]]] = [
8             [] for _ in range(n)
9         ]
10        self.indegree: List[int] = [0] * n
11
12    def add_edge(self, u: int, v: int, weight: float) -> None:
13        """添加有向边 u -> v, 权值为 weight, 并更新入度。"""
14        pass # TODO
15
16    def topological_sort(self) -> List[int]:
17        """
18        使用 Kahn 算法返回拓扑序。
19        注意使用 indegree 的副本，避免改变图的原始数据。
20        若图中存在有向环（处理的顶点数 < n），则抛出 ValueError。
21        """
22        pass # TODO
23
24    def shortest_paths(
25        self, source: int
26    ) -> Tuple[List[float], List[int]]:
27        """
28        先调用 topological_sort() 获取拓扑序，
29        再按拓扑序松弛所有边，返回 (dist, parent)。
30
31        步骤提示：
32        1. 调用 topological_sort()（若含环则抛出 ValueError）。
33        2. 初始化 dist 全为 inf, parent 全为 -1; dist[source]=0。
34        3. 按拓扑序遍历每个顶点 u:
35            - 若 dist[u] == inf, 跳过（源点不可达）；
36            - 遍历 u 的所有出边 (u, v, w)，执行松弛。
37        """
38        pass # TODO
```

```
39
40     @staticmethod
41     def reconstruct_path(
42         source: int, target: int, parent: List[int]
43     ) -> List[int]:
44         """恢复 source 到 target 的最短路径。不可达则返回 []。"""
45         pass # TODO
```

2.5 测试用例

```
1 if __name__ == "__main__":
2     # 情形 1: 标准 DAG 含负权边
3     dag = DAGShortestPath(7)
4     dag.add_edge(0, 1, 5)
5     dag.add_edge(0, 2, 3)
6     dag.add_edge(1, 3, 6)
7     dag.add_edge(1, 2, 2)
8     dag.add_edge(2, 4, 4)
9     dag.add_edge(2, 5, 2)
10    dag.add_edge(2, 3, 7)
11    dag.add_edge(3, 4, -1)
12    dag.add_edge(4, 5, -2)
13    # 顶点 6 不可达
14
15    dist, parent = dag.shortest_paths(0)
16    print(dist)
17    # [0, 5, 3, 10, 7, 5, inf]
18
19    print(dag.reconstruct_path(0, 5, parent))
20    # [0, 2, 5], 距离为 5
21
22    # 情形 2: 含环图不能使用 DAG Relaxation
23    cyclic = DAGShortestPath(3)
24    cyclic.add_edge(0, 1, 1)
25    cyclic.add_edge(1, 2, 1)
26    cyclic.add_edge(2, 0, 1)
27
28    try:
29        cyclic.shortest_paths(0)
```

```
30     except ValueError as error:
31         print(error) # 图中存在有向环
32
33     # 情形 3: 源点为中间节点
34     dag3 = DAGShortestPath(4)
35     dag3.add_edge(0, 1, 1)
36     dag3.add_edge(1, 2, 1)
37     dag3.add_edge(2, 3, 1)
38     dist3, _ = dag3.shortest_paths(1)
39     print(dist3) # [inf, 0, 1, 2]
40
41     # 情形 4: 单顶点
42     single = DAGShortestPath(1)
43     dist_s, _ = single.shortest_paths(0)
44     print(dist_s) # [0]
```

第 3 题 允许负权边的最短路径（Bellman-Ford）

考点：重复松弛 • 负权边 • 负权环检测 • 提前终止

3.1 背景知识

Bellman-Ford 算法适用于一般有向带权图，并允许边权为负数。算法连续进行至多 $|V| - 1$ 轮操作，每轮依次松弛所有边。

若从源点到顶点 v 的最短路径存在且不经过重复顶点，则该路径最多包含 $|V| - 1$ 条边。因此，经过 k 轮完整松弛后，所有最短路径边数不超过 k 的顶点都能得到正确距离。

算法主体可写为

$$\begin{cases} d[s] = 0, & d[v] = +\infty \quad (v \neq s), \\ d[v] \leftarrow \min\{d[v], d[u] + w(u, v)\}, & \forall (u, v) \in E, \\ \text{重复上述过程至多 } |V| - 1 \text{ 轮。} \end{cases} \quad (4)$$

完成 $|V| - 1$ 轮后，再检查所有边。若仍存在

$$d[u] \neq +\infty \quad \text{且} \quad d[u] + w(u, v) < d[v],$$

则说明存在一个从源点可达的负权环。沿该环不断行走会使路径权值无限减小，因此相关顶点不存在有限最短距离。

若某一轮没有任何距离更新，可以提前终止算法。最坏时间复杂度为 $O(|V||E|)$ ，空间复杂度为 $O(|V|)$ 。

3.2 引导性问题

1. 为什么算法最多需要执行 $|V| - 1$ 轮松弛？一条不包含环的简单路径最多有多少条边？
2. 检测负权环时，为什么必须要求 $d[u] \neq +\infty$ ？一个源点不可达的负权环是否会影响当前单源最短路径问题？
3. 若图中存在权值为正的环，为什么它不会导致最短距离不存在？
4. 某轮没有任何更新时，为什么可以立即结束算法？
5. Bellman-Ford 与 DAG Relaxation 都允许负权边，两者在图结构限制和时间复杂度方面有什么区别？
6. 如果检测到可达负权环，还能否直接使用当前 `parent` 数组恢复普通最短路径？

3.3 题目要求

实现以下方法：

1. `add_edge(u, v, weight)`：将边存入边列表。
2. `shortest_paths(source)`：执行 Bellman-Ford 算法。
3. 若存在源点可达的负权环，抛出 `ValueError`。
4. 加入“本轮是否发生更新”的标记，实现提前终止。
5. 使用前驱数组恢复不存在负权环时的最短路径。

3.4 接口参考（仅声明，补全实现）

```
1 from typing import List, Tuple
2
3 class BellmanFordGraph:
4     def __init__(self, n: int):
5         self.n = n
6         self.edges: List[Tuple[int, int, float]] = []
7
8     def add_edge(self, u: int, v: int, weight: float) -> None:
9         """添加有向边 u -> v，权值为 weight。"""
10        pass # TODO
11
12    def shortest_paths(
13        self, source: int
```

```

14     ) -> Tuple[List[float], List[int]]:
15         """
16         返回源点到各顶点的最短距离和前驱数组。
17
18         步骤提示：
19         1. 初始化 dist 全为 inf, parent 全为 -1; dist[source]=0。
20         2. 进行 n-1 轮松弛，每轮用 updated 标记实现提前终止。
21         3. 第 n 轮检测负权环：若仍可松弛且 dist[u]!=inf,
22            抛出 ValueError("存在从源点可达的负权环")。
23         """
24         pass # TODO
25
26     @staticmethod
27     def reconstruct_path(
28         source: int, target: int, parent: List[int]
29     ) -> List[int]:
30         """恢复 source 到 target 的最短路径。不可达则返回 []。"""
31         pass # TODO

```

3.5 测试用例

```

1 if __name__ == "__main__":
2     # 情形 1: 含负权边，但不存在负权环
3     graph = BellmanFordGraph(6)
4     graph.add_edge(0, 1, 6)
5     graph.add_edge(0, 2, 7)
6     graph.add_edge(1, 2, 8)
7     graph.add_edge(1, 3, 5)
8     graph.add_edge(1, 4, -4)
9     graph.add_edge(2, 3, -3)
10    graph.add_edge(2, 4, 9)
11    graph.add_edge(3, 1, -2)
12    graph.add_edge(4, 0, 2)
13    graph.add_edge(4, 3, 7)
14
15    dist, parent = graph.shortest_paths(0)
16    print(dist) # [0, 2, 7, 4, -2, inf]
17    print(graph.reconstruct_path(0, 4, parent))
18    # [0, 2, 3, 1, 4]

```

```
19
20 # 情形 2: 存在从源点可达的负权环
21 # 0 --(1)--> 1 --(-2)--> 2 --(-2)--> 1 (1<->2 负权环)
22 negative_cycle = BellmanFordGraph(4)
23 negative_cycle.add_edge(0, 1, 1)
24 negative_cycle.add_edge(1, 2, -2)
25 negative_cycle.add_edge(2, 1, -2)
26 negative_cycle.add_edge(2, 3, 1)
27
28 try:
29     negative_cycle.shortest_paths(0)
30 except ValueError as error:
31     print(error) # 存在从源点可达的负权环
32
33 # 情形 3: 负权环不可达, 不影响从源点 0 出发的结果
34 # 分量 A: 0 --(3)--> 1 --(4)--> 2 (无环)
35 # 分量 B: 3 --(-1)--> 4 --(-1)--> 3 (负权环, 但从 0 不可达)
36 unreachable_cycle = BellmanFordGraph(5)
37 unreachable_cycle.add_edge(0, 1, 3)
38 unreachable_cycle.add_edge(1, 2, 4)
39 unreachable_cycle.add_edge(3, 4, -1)
40 unreachable_cycle.add_edge(4, 3, -1)
41
42 dist, _ = unreachable_cycle.shortest_paths(0)
43 print(dist) # [0, 3, 7, inf, inf]
44
45 # 情形 4: 含负权边, 提前终止验证
46 bf4 = BellmanFordGraph(3)
47 bf4.add_edge(0, 1, 2)
48 bf4.add_edge(0, 2, 4)
49 dist4, _ = bf4.shortest_paths(0)
50 print(dist4) # [0, 2, 4]
51
52 # 情形 5: 单顶点
53 bf5 = BellmanFordGraph(1)
54 dist5, _ = bf5.shortest_paths(0)
55 print(dist5) # [0]
```

第 4 题 非负权图最短路径 (Dijkstra)

考点：贪心策略 • 优先队列 (最小堆) • 惰性删除 • 非负权限制

4.1 背景知识

Dijkstra 算法适用于所有边权均为非负数的图。算法维护源点到各顶点的距离估计，并反复选择当前距离最小的未确定顶点 u ，将 $d[u]$ 确定为最终最短距离，再松弛其出边。

贪心选择的正确性依赖非负权条件。设 u 是当前距离估计最小的未确定顶点，则任何经过其他未确定顶点再到达 u 的路径，都不可能使 $d[u]$ 进一步减小——因为所有边权均非负，绕路只会增加（或持平）路径长度。

使用最小堆时，可以将候选状态 $(d[v], v)$ 加入优先队列。由于堆不支持高效的“减键”操作，同一顶点可能多次入堆，因此出堆时应采用惰性删除策略：

$$\text{若 } d_{\text{heap}} > d[u], \text{ 则该条目已经过时，直接跳过。} \quad (5)$$

采用邻接表和二叉堆时，时间复杂度为

$$O((|V| + |E|) \log |V|),$$

对于连通图通常简写为 $O(|E| \log |V|)$ 。

4.2 引导性问题

1. Dijkstra 的贪心选择是什么？为什么最小距离顶点可以被确定？
2. 若图中存在负权边，已经确定的顶点距离是否可能再次减小？请构造一个反例。
3. 为什么 Python 的 `heapq` 中允许同一顶点多次入堆？
4. 什么是惰性删除？若不跳过过期状态，算法结果和运行效率会受到什么影响？
5. 为什么不能简单地用普通队列替代最小堆？
6. 如果边权全部等于 1，应优先使用 BFS 还是 Dijkstra？请从复杂度和实现难度两方面比较。

4.3 题目要求

实现以下方法：

1. `add_edge(u, v, weight)`：添加带权边；若权值为负，则抛出 `ValueError`。
2. `shortest_paths(source)`：使用最小堆 + 惰性删除实现 Dijkstra。

3. 使用 `parent` 数组恢复最短路径。
4. 通过参数 `undirected` 支持无向图和有向图。

4.4 接口参考（仅声明，补全实现）

```
1 import heapq
2 from typing import List, Tuple
3
4 class DijkstraGraph:
5     def __init__(self, n: int, undirected: bool = False):
6         self.n = n
7         self.undirected = undirected
8         self.adj: List[List[Tuple[int, float]]] = [
9             [] for _ in range(n)
10        ]
11
12    def add_edge(self, u: int, v: int, weight: float) -> None:
13        """
14        添加边 u -> v, 权值为 weight (必须 >= 0)。
15        若 undirected=True, 同时添加 v -> u。
16        若 weight < 0, 抛出 ValueError。
17        """
18        pass # TODO
19
20    def shortest_paths(
21        self, source: int
22    ) -> Tuple[List[float], List[int]]:
23        """
24        使用最小堆 + 惰性删除运行 Dijkstra,
25        返回 (dist, parent)。
26
27        步骤提示:
28        1. 初始化 dist 全为 inf, parent 全为 -1; dist[source]=0。
29        2. 将 (0, source) 推入最小堆。
30        3. 当堆非空时:
31            - 弹出 (d, u);
32            - 若 d > dist[u] (惰性删除), 跳过;
33            - 遍历 u 的所有出边 (u, v, w):
34                若 dist[u] + w < dist[v], 更新 dist[v] 和 parent[v],
```

```

35         并将 (dist[v], v) 推入堆。
36     4. 返回 dist, parent。
37     """
38     pass # TODO
39
40     @staticmethod
41     def reconstruct_path(
42         source: int, target: int, parent: List[int]
43     ) -> List[int]:
44         """恢复 source 到 target 的最短路径。不可达则返回 []。"""
45         pass # TODO

```

4.5 测试用例

```

1  if __name__ == "__main__":
2      # 情形 1: 无向图, 标准情形
3      # 0 --4-- 1 --1-- 3 --3-- 4 --2-- 5
4      # |      |    /      /
5      # 1      2    5      7
6      # |      |    /      /
7      # 2 -----+ /      /
8      #          1 -----+
9      g1 = DijkstraGraph(7, undirected=True)
10     g1.add_edge(0, 1, 4)
11     g1.add_edge(0, 2, 1)
12     g1.add_edge(2, 1, 2)
13     g1.add_edge(1, 3, 1)
14     g1.add_edge(2, 3, 5)
15     g1.add_edge(3, 4, 3)
16     g1.add_edge(1, 4, 7)
17     g1.add_edge(4, 5, 2)
18     # 顶点 6 不可达
19
20     dist, parent = g1.shortest_paths(0)
21     print(dist)
22     # [0, 3, 1, 4, 7, 9, inf]
23     print(g1.reconstruct_path(0, 5, parent))
24     # [0, 2, 1, 3, 4, 5]
25     print(g1.reconstruct_path(0, 6, parent))

```

```
26     # []
27
28     # 情形 2: 有向图, 存在更短的绕路
29     # 0 --(10)--> 1
30     # 0 --(1)--> 2 --(1)--> 1
31     g2 = DijkstraGraph(3)
32     g2.add_edge(0, 1, 10)
33     g2.add_edge(0, 2, 1)
34     g2.add_edge(2, 1, 1)
35     dist2, _ = g2.shortest_paths(0)
36     print(dist2) # [0, 2, 1]
37
38     # 情形 3: 非连通图, 部分顶点不可达
39     g3 = DijkstraGraph(4)
40     g3.add_edge(0, 1, 5)
41     g3.add_edge(2, 3, 2) # 与 0 不连通
42     dist3, _ = g3.shortest_paths(0)
43     print(dist3) # [0, 5, inf, inf]
44
45     # 情形 4: 单顶点
46     g4 = DijkstraGraph(1)
47     dist4, _ = g4.shortest_paths(0)
48     print(dist4) # [0]
49
50     # 情形 5: 稠密图, 验证惰性删除正确性
51     # 0 --(2)--> 1 --(2)--> 4
52     # 0 --(1)--> 2 --(1)--> 3 --(1)--> 4
53     # 0 --(5)--> 4
54     g5 = DijkstraGraph(5)
55     g5.add_edge(0, 1, 2)
56     g5.add_edge(1, 4, 2)
57     g5.add_edge(0, 2, 1)
58     g5.add_edge(2, 3, 1)
59     g5.add_edge(3, 4, 1)
60     g5.add_edge(0, 4, 5)
61     dist5, _ = g5.shortest_paths(0)
62     print(dist5)
63     # [0, 2, 1, 2, 3] (0->2->3->4 = 3 < 0->1->4 = 4 < 0->4 = 5)
64
65     # 情形 6: 负权边应被拒绝
```

```
66     try:
67         g1.add_edge(0, 6, -1)
68     except ValueError as error:
69         print(error) # Dijkstra 不允许负权边
```


综合练习：最短路径算法的选择与比较

对下列情形，选择最合适的算法，并说明选择依据及时间复杂度。

1. 社交网络中每条好友关系均视为一步，求用户 s 到其他用户的最少关系层数。
2. 工程任务具有严格的先后依赖关系，边权表示完成前一任务后对总成本的影响，权值可能为负数。
3. 汇率转换图中允许出现负权边，需要判断是否存在能够使成本不断下降的可达环。
4. 城市道路长度均为非负实数，需要在大型稀疏路网中计算某城市到其他城市的最短距离。
5. 一张有向图既含负权边，也可能含有向环，但已经确定不存在负权环。
6. 一张图的所有边权都等于 5。应当使用 BFS 还是 Dijkstra？若使用 BFS，最终边数距离应如何转换成实际权值距离？

算法选择表

请补全下表。

图的条件	推荐算法	能否处理负权边	时间复杂度
无权图	_____	_____	_____
DAG	_____	_____	_____
一般图，允许负权边	_____	_____	_____
一般图，边权非负	_____	_____	_____

提高题：统一最短路径求解器

设计函数

```
1 def choose_sssp_algorithm(  
2     n: int,  
3     edges: list,  
4     source: int,  
5     is_directed: bool,  
6     is_dag: bool,  
7     is_unweighted: bool  
8 ) -> tuple:  
9     """  
10    根据图的性质自动选择 BFS、DAG Relaxation、  
11    Bellman-Ford 或 Dijkstra。
```

```
12
13     返回:
14     algorithm_name, dist, parent
15     """
16     pass # TODO
```

算法选择规则如下：

1. 若图无权，使用 BFS。
2. 否则，若图是 DAG，使用 DAG Relaxation。
3. 否则，若存在负权边，使用 Bellman-Ford。
4. 否则，使用 Dijkstra。

请进一步思考：如果调用者没有直接提供 `is_dag`，程序应当如何自动判断一张有向图是否为 DAG？

附：编写建议与常见误区

1. 初始化距离。源点距离应设为 0，其余顶点应设为 `float("inf")`，不能全部初始化为 0。
2. 不可达顶点不能参与松弛。尤其在 Bellman-Ford 和 DAG Relaxation 中，应先判断 `dist[u] != float("inf")`。
3. BFS 需要在入队时标记顶点。若在出队时才标记，同一个顶点可能被重复加入队列。
4. DAG Relaxation 必须先验证无环。若拓扑排序得到的顶点数小于 n ，说明图中存在有向环，不能继续使用该算法。
5. Bellman-Ford 只报告源点可达的负权环。不可达分量中的负权环不会影响当前源点的最短路径。
6. Dijkstra 禁止负权边。负权边会破坏“当前最小距离已经确定”的贪心结论。
7. 最小堆中的同一顶点可以出现多次。出堆时使用 `if current_dist > dist[u]: continue` 跳过过期状态。
8. 路径恢复必须处理不可达情形。若目标点不是源点且其前驱始终为 -1 ，应返回空列表，不能错误地返回仅包含目标点的列表。
9. 相同最短距离可能对应多条路径。测试时应重点检查路径权值与最短距离是否一致，不应强制要求前驱数组完全相同。
10. 注意有向边与无向边的存储区别。无向边需要在邻接表中存储两个方向；有向边只存储题目给定的方向。
11. 比较算法时应同时说明适用条件。时间复杂度更低不代表算法在所有图上都适用。例如 DAG Relaxation 很快，但只能处理有向无环图。

建议完成顺序

1. 先完成 BFS，熟悉距离数组、前驱数组和路径恢复。
2. 再完成拓扑排序，并在此基础上实现 DAG Relaxation。
3. 完成 Bellman-Ford，重点检查提前终止与负权环检测。
4. 最后实现堆优化 Dijkstra，重点理解惰性删除。
5. 使用同一张非负权 DAG 分别运行后三种算法，比较它们输出的距离数组是否一致。